

Programmazione Web e Mobile con Laboratorio

Lezione 16 - Gestione login
(JWT e autenticazione)

Cookies vs sessioni

Aspetto	Cookie	Sessione
Dove è memorizzato	Sul client (browser, nei dati locali)	Sul server (in memoria, file, DB, ecc.)
Chi lo gestisce	Il browser, ma può essere letto/scritto dal server	Gestito dal server, spesso con un identificatore salvato nel cookie
Persistenza	Può essere persistente (es. expires=...)	Di solito temporanea (finché la sessione non scade)
Sicurezza	Il contenuto è visibile/modificabile dal client (se non firmato o criptato)	Sicura: i dati sensibili restano sul server
Usi tipici	Ricordare preferenze, token, dati non critici	Tracciare login utente, carrello, autenticazioni
Limiti	Max 4KB per cookie, max 20 cookie per dominio circa	Nessun limite pratico salvo risorse di sistema

Per il login

- Sessione:
 - il server crea una sessione (con un ID) quando l'utente fa login
 - salva nella sessione i dati dell'utente (es. userId, ruolo)
- Cookie:
 - il server invia un cookie contenente il session ID
 - Il session ID viene rinviato dal browser ad ogni richiesta
- Il cookie è solo un contenitore dell'identificatore, mentre i dati della sessione sono sul server

JSON Web Token (JWT)

- Un JWT è un token compatto, firmato e auto-contenuto
- Rappresenta in modo sicuro delle informazioni (ad esempio, l'identità di un utente)
- Viene spesso usato per gestire autenticazione stateless
- Alternativa più adatta per architetture RESTful

Struttura di un JWT

- Un JWT è una stringa divisa in tre parti separate da punti
 - HEADER.PAYLOAD.SIGNATURE
- Un esempio reale può somigliare a:
 - eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJkYXN0IiwiaWF0IjE1xIXKn8jx0M78D-Tm_Q

JWT Header

- È un oggetto JSON che descrive come è firmato il token

- Esempio:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- alg: l'algoritmo di firma
- typ: tipo di token
- Viene codificato con base64url in una stringa compatibile con le URL

JWT Payload

- Contiene le informazioni da trasmettere

- Esempio:

```
{  
  "userId": 123,  
  "role": "admin",  
  "exp": 1716103600  
}
```

- exp: timestamp di scadenza
- Anche questo è base64url encoded

JWT Signature

- Serve a verificare che il token non sia stato modificato
- Calcolata così:

```
HMACSHA256(  
    base64urlEncode(header) + "." +  
    base64urlEncode(payload),  
    secret  
)
```

- Il server firma il token con una chiave segreta
- Se il token viene modificato, la firma non combaccerà e sarà considerato non valido

Sessioni vs JWT

Caratteristica	Sessione (cookie-based)	JWT (token-based)
Stato lato server	Sì (memorizzata in RAM/Redis/DB)	No (stateless, il server non conserva nulla)
Scalabilità	Richiede session store centralizzato	Scalabile orizzontalmente
Autenticazione RESTful	Non aderente al principio stateless REST	Compatibile con REST API
Verifica lato server	Sessione trovata via ID in cookie	JWT verificato tramite firma
Trasmissione credenziali	Cookie (automatico nei browser)	Header Authorization: Bearer <token>
Client non browser (es. app mobile)	Cookie da gestire manualmente	Semplice: header standard
Gestione logout	Facile: distruggi la sessione	Difficile: serve blacklist o short expiry
Durata credenziali	Controllata dal server	Controllata da exp nel token

Creazione di un token

- HS256 è un algoritmo di firma simmetrica
- Non inserire mai dati sensibili nel payload perché è visibile a chiunque legga il token

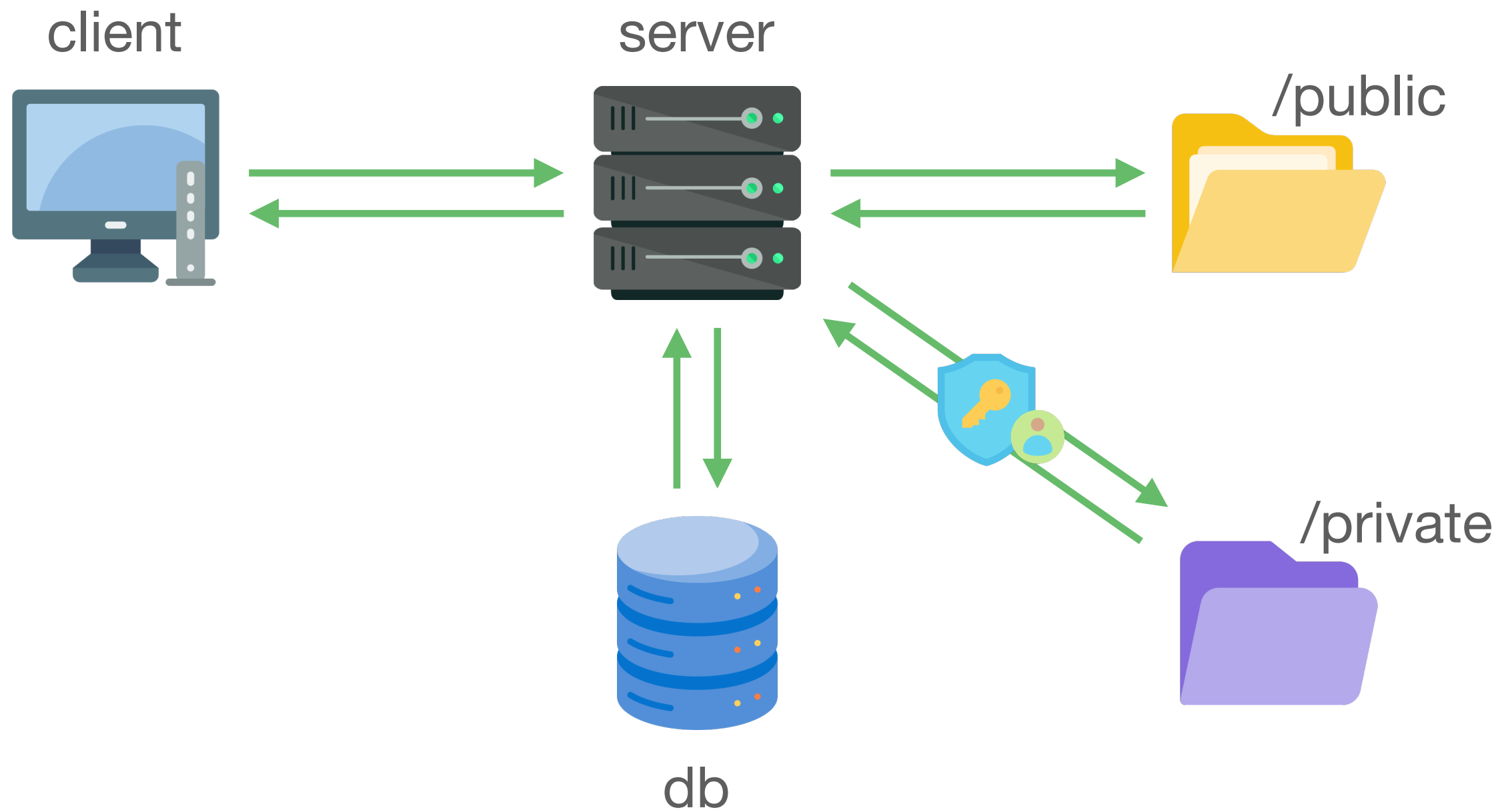
```
const jwt = require("jsonwebtoken");

const JWT_SECRET = "mia_chiave_super_segreta";

const payload = {
  userId: "user1",
  email: "user1@email.com",
};

const token = jwt.sign(payload, JWT_SECRET, {
  algorithm: "HS256",
  expiresIn: "1h"
});
```

Area riservata



Funzione di login

```
app.post("/api/login", async (req, res) => {
  const { email, password } = req.body;
  const query = "SELECT id, nome FROM utenti WHERE email = ? AND password = ?";
  try {
    const [righe, colonne] = await pool.promise().execute(query, [email, password]);
    const user = righe[0];
    const payload = {
      userId: user.id,
      userName: user.nome,
    };
    const token = jwt.sign(payload, JWT_SECRET, {
      algorithm: "HS256",
      expiresIn: "1h"
    });
    res.cookie("token", token, {
      httpOnly: true,
      secure: true,
      maxAge: 3600000,
      sameSite: "Strict"
    });
    return res.json({
      success: true,
      message: "Login riuscito"
    });
  }
  ...
});
```

Funzione di login

- `const payload = ...` prepara il payload da inserire nel JWT
- `jwt.sign(payload, JWT_SECRET, {...})` genera il token
 - firmato con la chiave `JWT_SECRET`
 - usando algoritmo e altri parametri specificati
- `res.cookie()` imposta un cookie con il token

Accesso a pagine protette

- Utente con token valido: accede ai file nella cartella /private
- Utente senza token o token invalido/scaduto: redirect

```
function authenticateToken(req, res, next) {  
  const token = req.cookies.token;  
  if (!token) {  
    return res.redirect(302, "/index.html");  
  }  
  try {  
    const payload = jwt.verify(token, JWT_SECRET);  
    req.user = payload;  
    next();  
  }  
  catch (err) {  
    return res.redirect(302, "/index.html");  
  }  
}  
  
app.use("/private", authenticateToken, express.static("private"));
```

Logout

- authenticateToken verifica se l'utente è autenticato
- res.clearCookie("token") cancella il cookie token dal browser
- equivale a fare logout, poiché senza token il client non potrà più accedere alle risorse protette

```
app.post("/api/logout", authenticateToken, (req, res) => {  
  res.clearCookie("token");  
  res.json({  
    success: true,  
    message: "Logout effettuato"  
  });  
});
```


Esercizio

Realizzare un'applicazione web composta da:

1. Server web basato su Node.js ed Express
2. Due tipi di risorse
 - Pagine pubbliche accessibili a tutti
 - Pagine private, accessibili solo dopo login
3. Login autenticato tramite JWT e memorizzato in cookie HttpOnly
4. Le credenziali dell'utente devono essere verificate tramite db

Problemi di sicurezza (1)

1. Furto del JWT: se il token è accessibile via JavaScript (es. localStorage), può essere rubato tramite attacchi XSS
 - Usare HttpOnly cookies, in modo che JavaScript non possa leggere il token
 - Usare Secure: true in produzione per trasmettere solo su HTTPS
2. Cross-Site Scripting (XSS): l'attaccante inietta codice JS malevolo che accede a dati sensibili
 - Sanitizzare sempre input e output HTML
 - Usare Content-Security-Policy lato server per limitare JS esterno
 - Usare HttpOnly per cookie critici (come il token JWT)

Problemi di sicurezza (2)

3. Cross-Site Request Forgery (CSRF): un sito esterno induce un browser autenticato a eseguire richieste non volute verso il tuo server
 - Attivare CSRF protection (es. token anti-CSRF)
 - Imposta i cookie con SameSite=Lax o SameSite=Strict per limitarne l'invio
4. Token leakage: un token può finire nei log, in URL, ecc.
 - Non mettere JWT nei parametri URL
 - Usa cookie HttpOnly oppure Authorization header

Problemi di sicurezza (3)

5. Token Replay: un token rubato viene riutilizzato per impersonare l'utente
 - Aggiungere scadenza brevi (expiresIn: "1h")
 - Considerare blacklist con log-out lato server
6. Manomissione Token: se la chiave di firma è debole o compromessa, un attaccante può firmare token falsi
 - Usare chiavi lunghe e segrete per HS256
 - Valutare RS256 per firmare con chiave privata (più sicuro per ambienti distribuiti)